



Khoa MẠNG MÁY TÍNH & TRUYỀN THÔNG - UIT

BÁO CÁO THỰC HÀNH

Thực hành Nhập môn Mạng máy tính

Lab 03: Hàm băm và chữ kí số

GVTH: Nguyễn Ngọc Truong

Ngày báo cáo: 27/04/2026

Nhóm 13 – NT101.Q22.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
23520536	Phạm Cao Minh Hoàng	Câu 1, câu 4	50%
24521388	Nguyễn Hoàng Phúc	Câu 2, câu 3	50%

1. Nhiệm vụ 2.1

- Phương pháp thực hiện:

- Sử dụng ngôn ngữ lập trình Python, ngôn ngữ cung cấp sẵn khả năng tính toán số nguyên lớn phù hợp để xử lý các phép toán mô-đun phức tạp của thuật toán RSA mà không bị lỗi tràn bộ nhớ.

- Mô tả một số hàm có trong thành phần chính:

- › **Nhóm hàm Toán học RSA (extended_gcd, modinv, calculate_keys):** Cốt lõi của chương trình. Áp dụng thuật toán Euclid mở rộng để tìm nghịch đảo mô-đun, qua đó tính toán Khóa riêng tư dựa trên Khóa công khai và phi hàm Euler.
- › **Nhóm hàm Sinh khóa ngẫu nhiên (is_prime, generate_random_rsa):** Cài đặt thuật toán kiểm tra tính nguyên tố Miller-Rabin để tự động tìm ra hai số nguyên tố lớn, phục vụ cho yêu cầu tạo bộ khóa ngẫu nhiên khi không được cung cấp tham số.
- › **Nhóm hàm Xử lý định dạng (int_to_text, is_valid_text):** Hỗ trợ chuyển đổi qua lại giữa mảng byte, số nguyên hệ cơ số 10 và văn bản UTF-8. Đặc biệt, hàm is_valid_text được thiết kế để loại bỏ các ký tự rác, chỉ chấp nhận các chuỗi văn bản có thể đọc được (printable).

- › **Hàm Giải mã (smart_decrypt):** Đây là hàm để giải quyết Câu 4. Hàm được thiết kế để tự động nhận diện bản mã là Khối đơn (Single block) hay Đa khối (Multi-block). Đồng thời, hàm tích hợp phương thức replace("\x00", "") để tước bỏ các byte rỗng (Null-byte padding) do thuật toán RSA đệm vào trong quá trình chia khối, giúp khôi phục chính xác thông điệp gốc.
- Yêu cầu mở đầu: Sinh bộ khóa ngẫu nhiên lớn nhất có thể:

a) Xác định khóa công khai PU và khóa riêng PR, nếu:

- $p_1 = 11, q_1 = 17, e_1 = 7$ (hệ thập phân)
- $p_2 = 20079993872842322116151219$
 $q_2 = 676717145751736242170789$
 $e_2 = 17$ (hệ thập phân)
- $p_3 = \text{F7E75FDC469067FFDC4E847C51F452DF}$,
 $q_3 = \text{E85CED54AF57E53E092113E62F436F4F}$,
 $e_3 = \text{0D88C3}$ (hệ thập lục phân)
- Đoạn code chương trình:

```
print("\n===== 1. TAO KHOA (YEU CAU 1) =====")
p1, q1, e1 = 11, 17, 7
PU1, PR1 = calculate_keys(p1, q1, e1)

p2, q2, e2 = 20079993872842322116151219, 676717145751736242170789, 17
PU2, PR2 = calculate_keys(p2, q2, e2)

p3 = int("F7E75FDC469067FFDC4E847C51F452DF", 16)
q3 = int("E85CED54AF57E53E092113E62F436F4F", 16)
e3 = int("0D88C3", 16)
PU3, PR3 = calculate_keys(p3, q3, e3)

print("PU1:", PU1, "PR1:", PR1)
print("PU2:", PU2, "PR2:", PR2)
print("PU3:", PU3, "PR3:", PR3)
```

- Phân tích code: đoạn mã sử dụng hàm calculate_keys với tác dụng:
- **Tác dụng:** Gộp gọn quy trình tạo khóa. Nhận vào p, q, e, tính toán Mô-đun $n = p * q$ trả về 2 cặp tuple đóng gói cần thận: Khóa công khai PU(e, n) và Khóa riêng PR(d, n).

- Kết quả thu được:

```
===== 1. TAO KHOA (YEU CAU 1) =====
PU1: (7, 187) PR1: (23, 187)
PU2: (17, 13588476140342208394395166469647627226674348541791) PR2: (7993221259024828467291262184080358019185876599873,
13588476140342208394395166469647627226674348541791)
PU3: (886979, 101776877529005912638346811918779931246783058062684819617574643018368103302097) PR3:
(24212225287904763939160097464943268930139828978795606022583874367720623008491,
101776877529005912638346811918779931246783058062684819617574643018368103302097)
```

⇒ Nhận xét:

Chương trình tính toán chính xác Mô-đun n và Khóa riêng d.

- Đối với Nhóm 1, khóa có kích thước rất nhỏ ($n = 187$).
- Đối với Nhóm 2 và Nhóm 3, chương trình xử lý tốt các số nguyên không lồ có độ dài hàng chục chữ số mà không gặp lỗi tràn số.

b) Sử dụng khóa được tạo ra từ p1,q1,e1 để mã hóa và giải mã bản rõ M=5 trong cả hai trường hợp Mã hóa cho tính bảo mật (EncryptionforConfidentiality) và Mã hóa cho tính xác thực(EncryptionforAuthentication).

- Đoạn code chương trình:

```
print("\n===== 2. MA HOA M = 5 (YEU CAU 2) =====")
M = 5
C = pow(M, PU1[0], PU1[1])
M_dec = pow(C, PR1[0], PR1[1])
S = pow(M, PR1[0], PR1[1])
M_auth = pow(S, PU1[0], PU1[1])

print("Confidentiality:", C, "->", M_dec)
print("Authentication:", S, "->", M_auth)
```

- Trong hàm này sử dụng hàm tối ưu có sẵn của hệ thống: Hàm pow(base, exp, mod)
 - **Tác dụng:** Để tính $C = M^e \bmod n$, thay vì viết $(M ** e) \% n$ (cách này sẽ làm tràn RAM máy tính nếu e quá lớn), hàm pow() thực hiện phép nhân bình phương có mô-đun ngay bên trong lời ngôn ngữ C.
- Kết quả thu được:

```
===== 2. MA HOA M = 5 (YEU CAU 2) =====
Confidentiality: 146 -> 5
Authentication: 180 -> 5
```

c) Sử dụng các khóa trên để mã hóa thông điệp sau: The University of Information Technology. Xác định bản mã dưới dạng Base64.

- Đoạn code chương trình:

```
# -----
# CAU 3: MA HOA CHUOI BASE 64
# -----
print("\n===== 3. MA HOA CHUOI (YEU CAU 3) =====")
msg = "The University of Information Technology"
msg_bytes = msg.encode()

e, n = PU2    # PHẢI dùng key lớn
block_in = 31
block_out = (n.bit_length() + 7) // 8
cipher_bytes = b""

for i in range(0, len(msg_bytes), block_in):
    chunk = msg_bytes[i:i+block_in]
    m_int = int.from_bytes(chunk, 'big')
    c_int = pow(m_int, e, n)
    cipher_bytes += c_int.to_bytes(block_out, 'big')

cipher_b64 = base64.b64encode(cipher_bytes).decode()
print("Cipher Base64:", cipher_b64)
```

- Phân tích code:

+Để giải quyết tình trạng độ dài của chuỗi lớn hơn độ dài của Mô-đun ($M > n$), code sử dụng các phương thức chuyển đổi dữ liệu sau:

- **encode() và int.from_bytes(chunk, 'big')**: Cắt chuỗi chữ ra thành các khối nhỏ (ví dụ 31 bytes), sau đó gom mảng byte này thành một con số để thuật toán RSA có thể tính toán được.
 - **to_bytes(block_out, 'big')**: Sau khi lấy con số đó đem mã hóa (bằng pow), kết quả sinh ra là một con số khác. Phương thức này sẽ dịch con số đó ngược lại thành mảng byte nhị phân thô, xác định kích thước block chuẩn để không bị lệch độ dài.
 - **base64.b64encode()**: Lấy toàn bộ mảng byte đã mã hóa đóng gói lại thành chuỗi văn bản Base64 theo yêu cầu của đề.
- **Kết quả chương trình:**

Cipher Base64: BWM0buy0XVeNfEaZU6F/C07YJ6RiBwe+hryuAyW7055WrzW6x547wioc

d) Tìm bản rõ tương ứng của mỗi bản mã sau, biết rằng chúng được mã hóa bằng 1 trong 3 khóa đã cho ở trên:

- Đoạn mã chương trình:

[illegible]

- **Phân tích code:**

+ Chuẩn bị Dữ liệu và Tập khóa (Data & Key Preparation)

- Chương trình định nghĩa mảng ciphertexts chứa 4 đoạn bản mã. Mỗi phần tử là một Tuple gồm 3 thông tin: Tên đoạn mã, Chuỗi dữ liệu thô, và Định dạng mã hóa (base64, hex, hoặc binary).
- Mảng all_keys tập hợp 3 bộ khóa (bao gồm cả Public Key và Private Key) đã được tính toán ở các yêu cầu trước.

+Vòng lặp for name, data, mode in ciphertxts đảm nhiệm vai trò phiên dịch:

- Nếu định dạng là base64: Gọi hàm `base64.b64decode()` để giải mã thành mảng byte thô.
- Nếu định dạng là hex: Gọi hàm `bytes.fromhex()` để chuyển chuỗi thập lục phân thành mảng byte.

- Nếu định dạng là binary: Ép kiểu chuỗi nhị phân hệ cơ số 2 thành số nguyên (`int(data, 2)`), sau đó chuyển số nguyên này thành mảng byte với độ dài bit tương ứng.
- *Kết quả*: Dù đầu vào là chuẩn định dạng nào, đầu ra luôn được quy chuẩn về mảng byte (`c_bytes`) để chuẩn bị cho thuật toán giải mã.

+ Brute-force Decryption:

- **Thử giải mã Bảo mật**: Đưa mảng byte và Khóa riêng tư (`PR[0]`, `PR[1]`) vào hàm `smart_decrypt`.
 - **Thử giải mã Xác thực**: Đưa mảng byte và Khóa công khai (`PU[0]`, `PU[1]`) vào hàm `smart_decrypt`.
 - Biến cờ found được sử dụng để theo dõi trạng thái. Nếu hàm `smart_decrypt` (với khả năng xử lý chia khối và loại bỏ ký tự đệm `\x00`) trả về một mảng chứa văn bản hợp lệ, chương trình sẽ in kết quả và bật cờ `found = True`.
 - Nếu sau khi vét cạn toàn bộ 3 chìa khóa x 2 cơ chế (tổng cộng 6 trường hợp cho mỗi đoạn mã) mà không thu được văn bản có nghĩa, chương trình mới kết luận "Khong giai duoc".
- Kết quả thu được với việc thành công giải mã CT3:

```
===== 4. GIAI MA (YEU CAU 4) =====

--- Giai ma CT1 ---
Khong giai duoc

--- Giai ma CT2 ---
Khong giai duoc

--- Giai ma CT3 ---
[Key1] Bao mat: ('Vietnam National University - Ho Chi Minh City', 46)

--- Giai ma CT4 ---
Khong giai duoc
```

HẾT